nats

A command line utility to interact with and manage NATS.

This utility replaces various past tools that were named in the form `nats-sub` and `nats-pub`, adds several new capabilities and supports full JetStream management.

Check out the repo for all the details: github.com/nats-io/natscli↗.

Installing `nats`

Please refer to the installation section in the readme↗.

You can read about execution policies here↗.

Binaries are also available as GitHub Releases↗.

Using `nats`

Getting help

- NATS Command Line Interface README↗
- `nats help`
- `nats help [<command>...]` or `nats [<command>...] --help`

- Remember to look at the cheat sheets!
  - `nats cheat`
  - `nats cheat --sections`
  - `nats cheat <section>>`

Interacting with NATS

- `nats context`
- `nats account`
- `nats pub`
- `nats sub`
- `nats request`
- `nats reply`
- `nats bench`

Monitoring NATS

- `nats events`
- `nats rtt`
- `nats server`
- `nats latency`
- `nats governor`

Managing and interacting with streams

- `nats stream`
- `nats consumer`
- `nats backup`
- `nats restore`

Managing and interacting with the K/V Store

- `nats kv`

Get reference information

- `nats errors`
- `nats schema`

Configuration Contexts

The CLI has a number of configuration settings that can be passed either as command line arguments or set in environment variables.

```
nats --help
```

Output extract

```
...
  -s, --server=URL               NATS server urls ($NATS_URL)
      --user=USER                Username or Token ($NATS_USER)
      --password=PASSWORD        Password ($NATS_PASSWORD)
      --creds=FILE               User credentials ($NATS_CREDS)
      --nkey=FILE                User NKEY ($NATS_NKEY)
      --tlscert=FILE             TLS public certificate
($NATS_CERT)
      --tlskey=FILE              TLS private key ($NATS_KEY)
      --tlsca=FILE               TLS certificate authority chain
($NATS_CA)
      --socks-proxy=PROXY        SOCKS5 proxy for connecting to
NATS server
                                 ($NATS_SOCKS_PROXY)
      --colors=SCHEME            Sets a color scheme to use
($NATS_COLOR)
      --timeout=DURATION         Time to wait on responses from
NATS
                                 ($NATS_TIMEOUT)
      --context=NAME             Configuration context
($NATS_CONTEXT)
...
```

The server URL can be set using the `--server` CLI flag, or the `NATS_URL` environment variable, or using NATS Contexts.

The password can be set using the `--password` CLI flag, or the `NATS_PASSWORD` environment variable, or using NATS Contexts. For example: if you want to create a script that prompts the user for the system user password (so that for example it doesn't appear in `ps` or `history` or maybe you don't want it stored in the profile) and then execute one or more `nats` commands you do something like:

```
#!/bin/bash
echo "-n" "system user password: "
read -s NATS_PASSWORD
export NATS_PASSWORD
nats server report jetstream --user system
```

NATS Contexts

A context is a named configuration that stores all of these settings. You can designate a default context and switch between contexts.

A context can be created with `nats context create my_context_name` and then modified with `nats context edit my_context_name`:

```
{
  "description": "",
  "url": "nats://127.0.0.1:4222",
  "token": "",
  "user": "",
  "password": "",
  "creds": "",
  "nkey": "",
  "cert": "",
  "key": "",
  "ca": "",
  "nsc": "",
  "jetstream_domain": "",
  "jetstream_api_prefix": "",
  "jetstream_event_prefix": "",
  "inbox_prefix": "",
  "user_jwt": ""
}
```

This context is stored in the file `~/.config/nats/context/my_context_name.json`.

A context can also be created by specifying settings with `nats context save`

```
nats context save example --server nats://nats.example.net:4222 -
-description 'Example.Net Server'
nats context save local --server nats://localhost:4222 --
description 'Local Host' --select
```

List your contexts

```
nats context ls
```

```
Known contexts:

   example             Example.Net Server
   local*              Local Host
```

We passed `--select` to the `local` one meaning it will be the default when nothing is set.

Select a context

```
nats context select
```

Check the round trip time to the server (using the currently selected context)

```
nats rtt
```

```
nats://localhost:4222:

   nats://127.0.0.1:4222: 245.115µs
       nats://[::1]:4222: 390.239µs
```

You can also specify a context directly

```
nats rtt --context example
```

```
nats://nats.example.net:4222:

   nats://192.0.2.10:4222: 41.560815ms
   nats://192.0.2.11:4222: 41.486609ms
   nats://192.0.2.12:4222: 41.178009ms
```

All `nats` commands are context aware and the `nats context` command has various commands to view, edit and remove contexts.

Server URLs and Credential paths can be resolved via the `nsc` command by specifying an URL, for example to find user `new` within the `orders` account of the `acme` operator you can use this:

```
nats context save example --description 'Example.Net Server' --
nsc nsc://acme/orders/new
```

The server list and credentials path will now be resolved via `nsc`, if these are specifically set in the context, the specific context configuration will take precedence.

Generating bcrypted passwords

The server supports hashing of passwords and authentication tokens using `bcrypt`. To take advantage of this, simply replace the plaintext password in the configuration with its `bcrypt` hash, and the server will automatically utilize `bcrypt` as needed. See also: Bcrypted Passwords.

The `nats` utility has a command for creating `bcrypt` hashes. This can be used for a password or a token in the configuration.

```
nats server passwd
```

```
? Enter password [? for help] **********************
? Reenter password [? for help] **********************

$2a$11$3kIDaCxw.Glsl1.u5nKa6eUnNDLV5HV9tIuUp7EHhMt6Nm9myW1aS
```

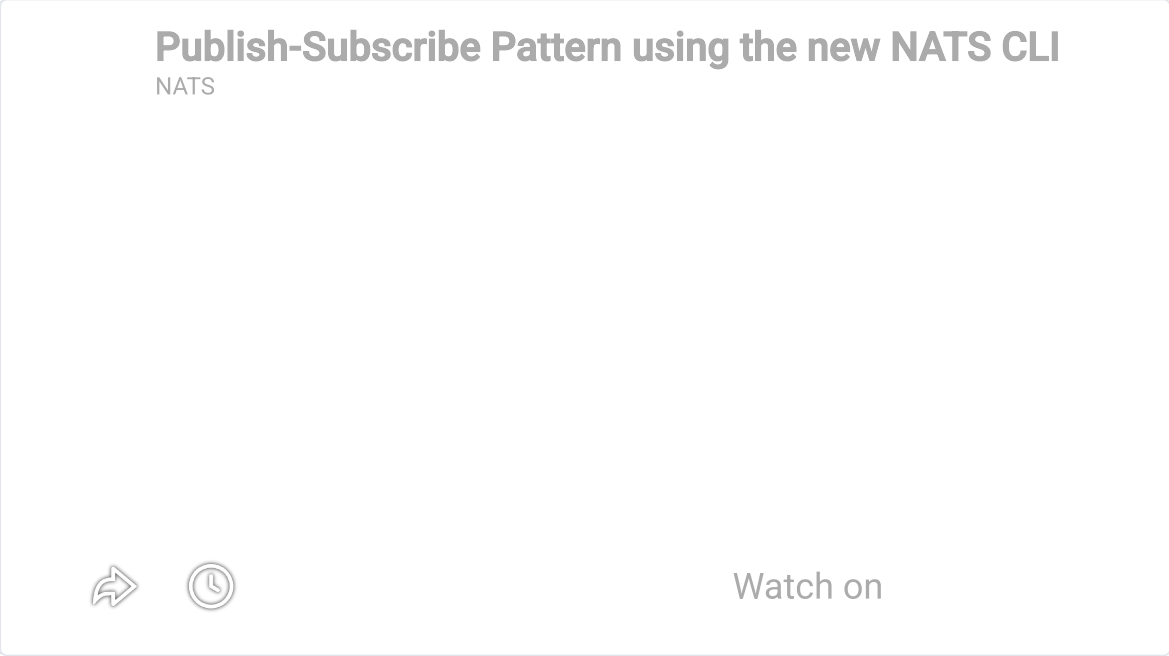
To use the password on the server, add the hash into the server configuration file's authorization section.

```
  authorization {
    user: derek
    password:
$2a$11$3kIDaCxw.Glsl1.u5nKa6eUnNDLV5HV9tIuUp7EHhMt6Nm9myW1aS
  }
```

Note the client will still have to provide the plain text version of the password, the server however will only store the hash to verify that the password is correct when supplied.

See Also

Publish-subscribe pattern using the NATS CLI



Publish-subscribe Pattern using NATS CLI

Previous
NATS Tools

Next
nats bench

Last updated 1 year ago

Was this helpful?